

Lab 4: Verification of timed automata

1/9/2026

Attempt 1



In Progress

NEXT UP: Submit Assignment



Add Comment

Unlimited Attempts Allowed

Details

Introduction

See also the lab introduction slides, which can be found [lab4-intro.pdf \(https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1\)](https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1). [↓ \(https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1\)](https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1)

Assignment Goals

The goal of this assignment is to provide initial hands-on experience with formal modeling and verification of real-time systems. This is done using the

Working in Groups

Solve this assignment in your groups. All students participating in the group shall be able to describe all parts of the solution.

land-in

A report, together with all the model and query files you have made, has to be submitted. The report must consist of answers to all parts of this assignment.

UPPAAL

UPPAAL is a tool for modeling, simulating, and verifying (via model checking) timed automata. Unless you have already used it earlier, you probably will not. You can find more information about UPPAAL at [the UPPAAL website](https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1) (<https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1>)

Please do not submit any report for the tutorial. The tutorial is there to help you; it is not part of the assignment.

You might also check out the [home page of UPPAAL](https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1) (<https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1>) for further documentation and help.

UPPAAL is installed on the department's Linux machines. You start it by running the command `uppaal` from a terminal.

UPPAAL is also installed in the virtual machine image provided for this course (see instructions in previous assignments about how to use the virtual machine).

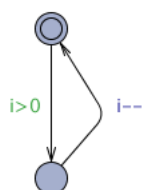
You can also run UPPAAL on your own machine. Download and install the academic version of UPPAAL from [the UPPAAL website](https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1) (<https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1>)

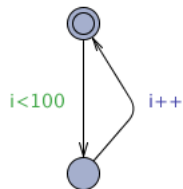
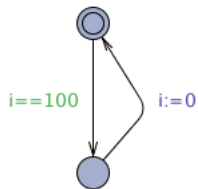
In case the above installation doesn't work, you could try downloading a copy of the tool here: [uppaal-4.0.13.7z \(https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1\)](https://uppsala.instructure.com/courses/108203/files/9564867?wrap=1). Extract the archive, rename the file to `uppaal.jar` in the root folder of the tool, `server`, and `verifyta` in the sub-folder called `bin-Linux`. After that, cd to the root folder containing `up`

Part 1: Verification Warm-Up

The main purpose of this part is to familiarize yourself with the UPPAAL tool and with its basic model-checking abilities.

Model the following three automata as three (separate) processes in Uppaal. The global variable `i` is of type integer.





To get a feeling for the behavior of the system, try to simulate it. Then answer the following questions using verification:

- Can the value of the variable i become strictly greater than 200?
- Is it true that no matter what happens (always) is the value of i smaller than or equal to 100?
- Is it true that no matter what happens (always) is the value of i strictly smaller than 100?
- Is there a run of the system in which i never exceeds 42?
- Is it true that no matter what happens (always) is the value of i greater than or equal to 0?

What to report:

The answers to the questions, together with the queries used for determining the answer.

Part 2: Scheduling

Problem introduction

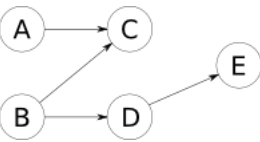
In this part, we will model a variant of the well-studied multiprocessor scheduling problem, the so-called "task graph scheduling on heterogeneous multi-processor systems". We consider two extensions of the basic model:

1. Jobs have dependency constraints. Some jobs may only be ready for execution after other jobs finished. Our schedule should honor these constraints.
2. Each job has a specific execution time on each processor. This means that the execution times on different processors can be different. Imagine, for example, that a job is faster on the first processor. Others may need to emulate FP instructions and may thus finish earlier on the second processor even with a lower clock rate.

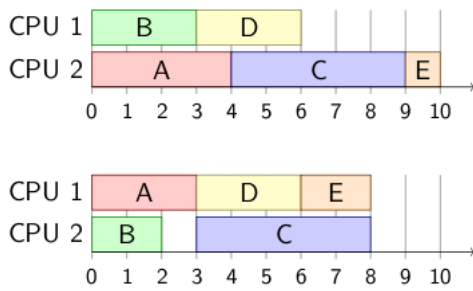
Formally, each job J is modeled as a tuple (C_1, ..., C_M) containing the execution times of that job on each of the M processors. Additionally, each job J is modeled as a set of dependencies D_J.

JOB	C_1	C_2	WAITS FOR
A	3	4	-
B	3	2	-
C	6	5	A, B
D	3	3	B
E	2	1	D

The example contains five jobs A to E with dependency constraints and different execution times on the two available processors. The dependencies are as follows:



There are many different ways in which these jobs can be scheduled, for example:



The first schedule takes 10 time units to complete, the second one only 8. It is easy to see that the second schedule must be optimal, since jobs A and C are in the critical path.

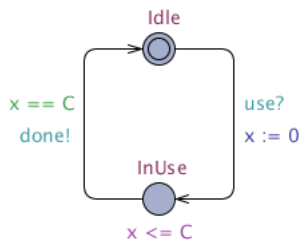
We will now use UPPAAL to model this problem in order to find optimal schedules and to prove their optimality.

Simple UPPAAL model

As a first approach, we simplify the problem and ignore the dependency constraints for now. The UPPAAL model will contain two different types of time

1. One automaton for each job
2. One automaton for each CPU

Jobs communicate with the CPUs via channels in order to "use" them. The CPU automaton (template) looks as follows:



Initially, the CPU is idle and waits for a job to request its use. When this happens, the CPU resets local clock x and waits in the **InUse** location until the CPU. Thus, the "Parameters:" field of the template should contain:

```
chan &use, chan &done, int &C
```

Using that, we can instantiate two CPUs in the "System declarations" as follows:

```
CPU1 = CPU(CPU1_use, CPU1_done, CPU1_C);
CPU2 = CPU(CPU2_use, CPU2_done, CPU2_C);
```

In order for this to work, the global declarations need to contain the channels and variables:

```
int CPU1_C;
chan CPU1_use, CPU1_done;
int CPU2_C;
chan CPU2_use, CPU2_done;
```

Enter the template "CPU" into UPPAAL together with the global and system declarations mentioned above. Based on this, your first task now is to model the appropriate channel. When it succeeds in doing so, it needs to tell the CPU how long the execution time will be (via writing the value to the global variable). Your template needs to be instantiated in the "System declarations" just as we instantiated the CPUs above. Further, you need to define the system as

```
system JobA, JobB, JobC, JobD, JobE, CPU1, CPU2;
```

Before we can try out the model, we need to complete it with an important detail. We would like to measure the overall execution time of the schedule. We can now finally express the property we want: What is the minimum value of clock "y" with "done_counter" being equal to the number of jobs

In order to answer this question, use the verifier with an appropriate reachability query. Setting the "Diagnostic Trace" option to "Fastest" will produce a

Extended UPPAAL model

Now, we want to express the dependencies between the jobs. Extend your model with a mechanism such that jobs respect the dependencies, i.e., the
When you are done, try again the above example. Does the generated trace correspond to the optimal schedule above? Further, generate queries to t

What to report:

- The "Job" template for the extended model, including a short description of what you did and why.
- All queries you used, i.e., for generating the trace as well as for proving optimality of the result. Please include a short description as well.
- Minimal execution time for the following problem instance:

JOB ↕	C_1 ↕	C_2 ↕	WAITS FOR ↕
A	1	2	-
B	4	2	-
C	5	5	A
D	2	4	A, B
E	3	7	B
F	7	8	C
G	1	1	C, D
H	5	2	G, E
I	3	1	E

Part 3: A Deadlock

In this last part of the assignment we will see how to model concurrent systems and how to detect subtle bugs in their design. As a basis for this, we lo
Let's revisit the problem: Three processes are working together where one is the Buffer that stores values produced by the Producer and later consum
should finish consuming the values as soon as a certain sum is reached. It is then its duty to tell the other two processes to stop working.

Some of the details which you had to implement in Ada we abstract away now. In particular, we assume that all values for the Buffer are the same (20,
code of the three processes you implemented in Ada stripped down to only the key functionality may look as follows:

```
// Buffer
while true:
  select
    when (items > 0) =>
      accept get;
      items--;
    or
      when (items < 3) =>
        accept put;
        items++;
    or
      accept finish;
      break;
  end select;
end while;

// Producer
while true:
  select
    delay random(0..3);
    Buffer.put;
  or
    accept finish;
    break;
```

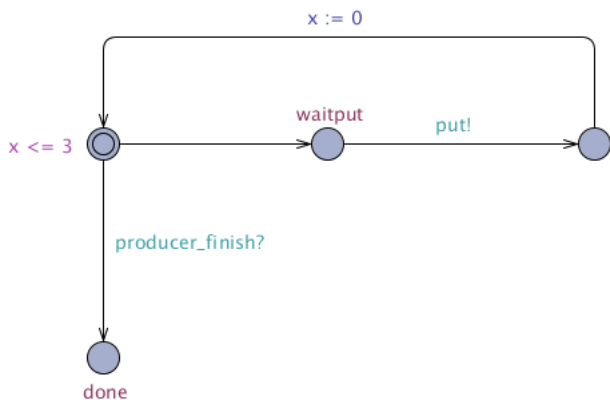
```

end select;
end while;

// Consumer
while true:
    delay random(0..3);
    if sum < 100:
        Buffer.get;
        sum := sum + 20;
    else:
        break;
end while;
Buffer.finish;
Producer.finish;

```

We would like to model the above pseudo-code in UPPAAL. As an example, the Producer may look as follows:



Look at how we modeled different aspects of the code like the random delay or the choice between delay and accepting the signal. Note that we prefix may need to declare local variables (like sum, items, clocks) and global ones (like channels). The "last" location of each automaton should be called "c automaton for the Producer.

First, simulate the system for a number of steps and observe how it behaves. You may see that if everything went fine, all three processes are in their A desired property of such a concurrent system is that it should never deadlock. We would like to verify this property which is usually done with the foll

$A \Box \text{not deadlock}$

Since all processes being in the "done" location formally constitutes a deadlock (no process can move further), we need to change the query in order t You will notice that the system contains a "real" deadlock, apart from the artificial one you excluded (all processes in "done"). Examine the problem in t for the Producer in order to terminate it, the Producer is waiting for the Buffer to accept a new item, but the Buffer is already terminated. (In Ada this w An obvious solution seems to be to first terminate the Producer and then the Buffer. Do the change to the Consumer automaton and run the verificatio

What to report:

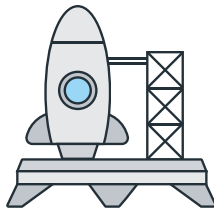
- The two timed automata for Consumer and Buffer
- The extended query to exclude the "artificial" deadlock state
- An explanation to the second deadlock
- Your suggested fix to that problem

Choose a submission type

Upload

Canvas Studio

More



[Choose a file to upload](#)

or

 Webcam Photo

 Canvas Files

Submit Assignment